



Query processing (and relational algebra too)

Content

✓ Steps in query processing (ch.15)

diagram on cost-based, don't mention anything about rule-based, but actually, the optimizer can be rule-based

optimizer choose best plan

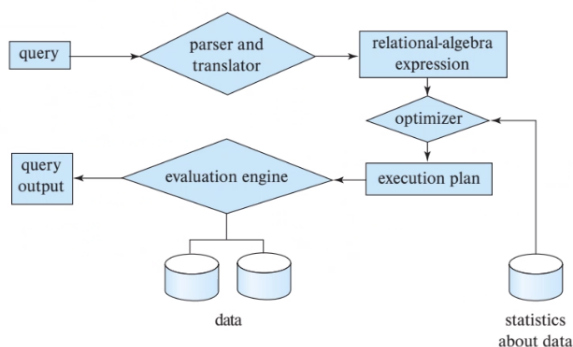


Figure 15.1 Steps in query processing.

- we have query input (we assume relational database diagram)
- the query arrived
- parser → check syntax
- translator → translate input query into a equivalent relational algebra expression(s).
- query optimization? will decide which relational algebra is best root? (refer to database statistics, cost based-rule based)
 - check which relational algebra statement will lead to best performance
 - size of table, size of row, number of row return for each condition.
- execution plan (the best plan) → see what best

- plan is kept in system catalog, so when similar query write, the same plan can be employ
- evaluation engine → follow execution plan based on relational algebra selected. simply execute relational algebra statement that selected
 - (advance ver?) we can implement in hardware level? we have the technology ?
- after evaluation, we have query output.

concentrate on cost-based optimization

- last time we use rule-based optimization on oracle (more than 30 years ago)
 - when oracle use oracle 6, and after that since oracle 7, oracle adopt cost-based optimizer.

Most of performance issues are handle by the DBMS.



Input query

- sql statement

```
select salary
from instructor
where salary < 75000;
```



After passing translation, sql statement translate into 2 algebra statement (relational-algebra expression)

- $\sigma_{salary < 75000} (\Pi_{salary} (instructor))$
- $\Pi_{salary} (\sigma_{salary < 75000} (instructor))$



Query Processing and Optimization

Generations of Programming language

1) **Machine language** (when we first develop computer the first language that we used is machine language)

- binary (set up switch → on and off and enter line by line)
- hexadecimal (later hexadecimal keyboard was invented)

2) **Assembly language** (after that for more readable code → assembly is invented)

- we have to know computer architecture, register, etc.

3) **High-level programming languages (3 GL)** (when you work with data like data files, the primitive command that come with high level language is call → record-at-a-time language) — **for example 1 get statement → you have 1 record or 1 object instance, 1 put → you write 1 record or object instance)**

- programmers write codes to search for the results which must be correct and also efficient (best response time)— we know that only correct result from request is already hard, efficient is another issues
 - if we load all this to programmer, it won't really be productive (they should focus more on correct)

4) **Set-oriented language (4GL)**

- one command work on set of instances at a time
- E.g. SQL, relational algebra, relational calculus, QUEL (competitor of sql), SEQUEL, QBE, etc.

There are 2 kinds

- Non procedural (declarative)
 - we declare what we want
- Procedural
 - how to obtain the result

Non procedural, procedural is still debate

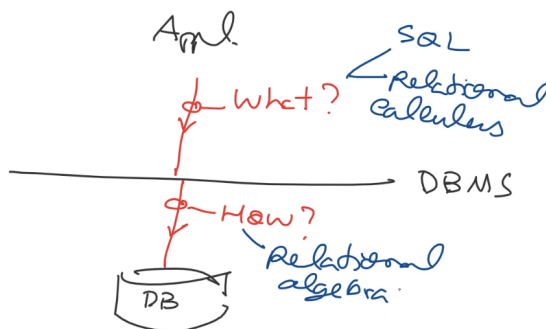
- SQL actually less procedural than c or relational algebra

5) Natural language or Formal natural language (5GL)

- use english and have the result come out
- maybe chatgpt can consider 5GL, assistance programmer in sql (you have to check)

More on number 4 (4GL)

- what and how?



Relational algebra (ch. 2)

Relational algebra is a relational database language



Relational Algebra

- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.
- Six basic operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - rename: ρ
- a relational database language (it means its only data structure is relation) (only one data structure: relation)
 - relation $\rightarrow$ the subset of cartesian product of domain

- input and output must be relation
- procedure language with several operators (it mean it has step like step 1, 2, 3)
 - several operators* (8 operator — in some book it said 6, because some can be derive from other, but let stick with 8, different book can said different thing)
 - **8 operators**
 - 1) select
 - 2) project
 - 3) join (natural join)

the first 3 combine, there are collectively call the essential subset

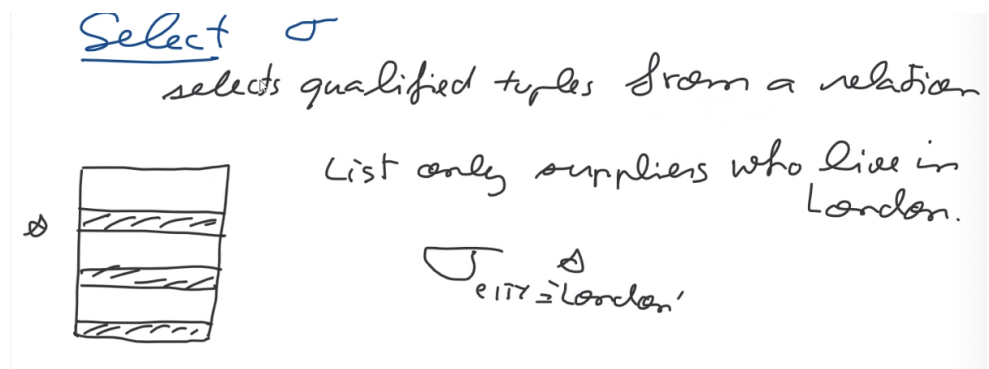
 - 4) union
 - 5) intersection
 - 6) difference
 - 7) product (cartesian product)
 - 8) divide

1)	Select	σ
2)	Project	π
3)	join	$\bowtie$
4)	Union	$\cup$
5)	intersection	$\cap$
6)	difference	$-$
7)	product	$\times$
8)	divide	$\div$

Select

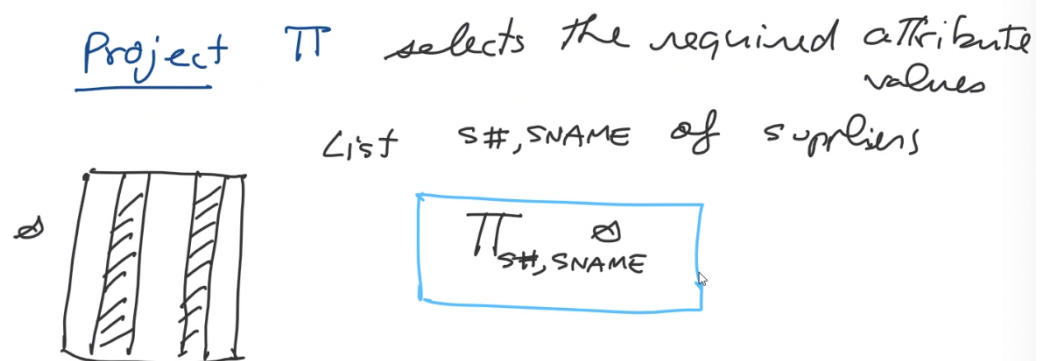
- the relational algebra is older than sql
 - it born with relational language and relational theory
 - around 10 years older than sql
 - so the word select of relational algebra come before sql select and it not the same

- it use the greek algebra small σ .
- select qualified tuples from a relation
- suppose we have table (or relation)
 - you may said you want to list only supplier who live in london.
 - the relational alebra woould be someting like
 - greek letter S supplier σ (where city = london)



Project (π)

- selects the required attribute values
- we want only some column
 - List S#, SNAME of suppliers



Ex: List s#, sname of suppliers who live in london

- here, to write relational algebra statement, we know that we choose see first who live in london $\rightarrow$ then S#, SNAME

Ex List S#, SNAME of suppliers
who live in London

$\Pi_{S\#, SNAME}(\sigma_{CITY='London'})$

- do inside parenthesis first (will be execute first)

Continue

SQL statements

*select salary
from instructor
where salary < 75000;*

- $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$
- $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$



must be table call instructor

- this can be translate into 2 possible relational algebra statement
 - first one → project from instructor table only salary column (only 1 column left, then screen out only those row whose salary is less than 75000)
 - second one → instead of using project first, we select salary less than 75000 first, and then after we have require row, we pick only salary column

this 2 are equivalent, and choose give the same result

- nowadays, when we keep database by row

- physical data is in database block → 1 database block comprise many os block, in os block we have row(s?)
- some dbms have limitation that row cannot be larger than 1 database block

Which one is better?

we simply keep data in a row, so we should search row first → we have index to help us search those row

- most dbms go to the lower one, because we keep data by row

if we choose the upper one, we have to go to all rows to obtain column, not really make much sense.

- nowadays, there are some analytics database, that keep data by column, but most dbms keep data by row(especially transaction one)
 - like finding mean to particular column (as column-based) for easy processing of build-in function — there are purpose built.

 Query-evaluation plan

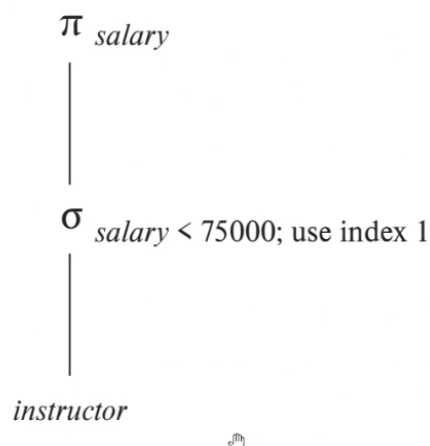


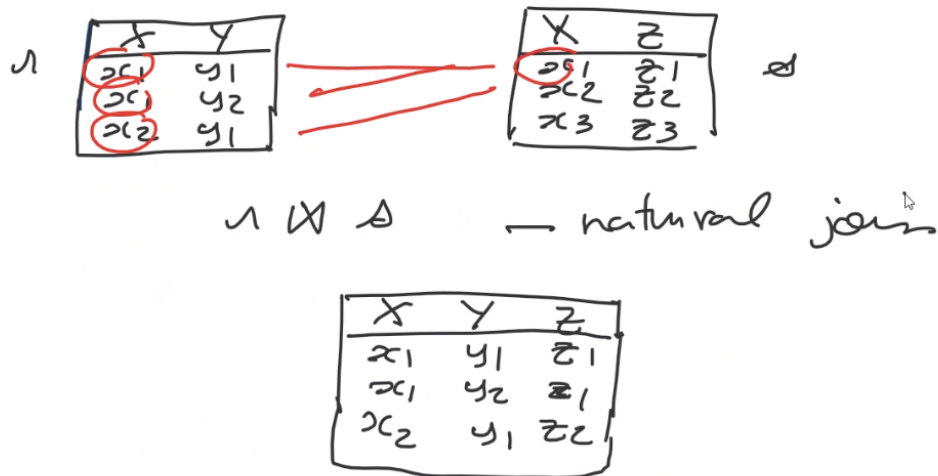
Figure 15.2 A query-evaluation plan.

- this can be retrieve from the database, most dbms has a tool to show the plan, oracle and postgres have tool that show plan in relational algebra statement

(cont)

Join (natural join) (symbol without comparison operator)

- this will match common value



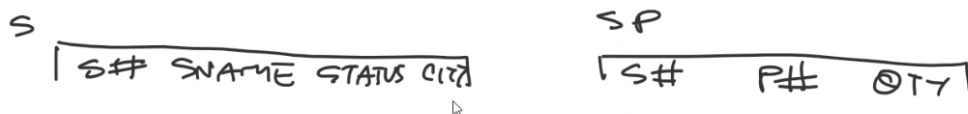
- a variation of join is called the theta join
 - the word theta refer to the join condition, which could something like =, >, <, ≥,
 - if you said r theta join s with equal join, the result will be (inner join, equi join)

a variation of join is called the theta join

$\bowtie_{=}$



Example (1)



List SNAME of London suppliers who supply P2. (Please write at least 2 relational algebra statements)

Relational Algebra:

- join first then select & project
- select & project before the join

$$\pi_{SNAME} (\sigma_{CITY='London' \wedge P\#='P2'} (S \bowtie P))$$

$$-\pi_{SNAME} \left(\left(\pi_{S\#, SNAME} (\sigma_{CITY='London'} S) \right) \bowtie \left(\pi_{S\#} (\sigma_{P\#='P2'} P) \right) \right)$$

- always use small letter, attribute name → capital letter.
- among this, query optimizer will find the best way.

Which one should be a better choice/plan

- if need database statistics, etc. to determine, as this point you can assume your statistic.
- which one should be a better plan

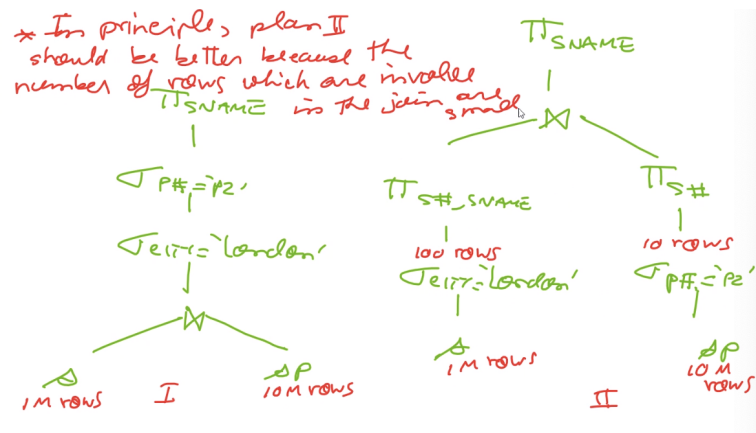
$$I - \pi_{SNAME} (\sigma_{CITY='London' \wedge P\#='P2'} (S \bowtie P))$$

$$II - \pi_{SNAME} \left(\left(\pi_{S\#, SNAME} (\sigma_{CITY='London'} S) \right) \bowtie \left(\pi_{S\#} (\sigma_{P\#='P2'} P) \right) \right)$$

Linear format of relational algebra statements

- Linear format is hard to comprehend.
- we may turn this linear format into a tree format.

Plan I and Plan II representation

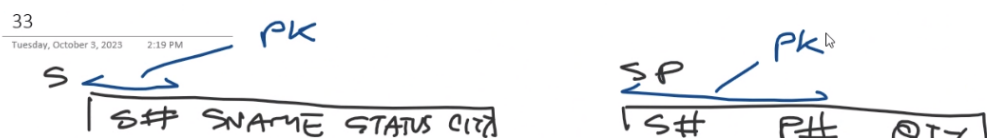


- which one is better depend on physical property (access thing)
- in theory, plan number 2 should be better, the idea is that we try to reduce the number of tuple to be join
 - join is a binary operator (2 side)
 - and if s has something like 1 million rows, sp is something like 10 million rows, joining 1 million to 10 million rows it hard (it take time)
 - in plan2, after the selection, those who have city = 'london' could be something like 1 M rows, and p2 could be just 10 rows.

idea → reduce number of row before joining (binary operation)

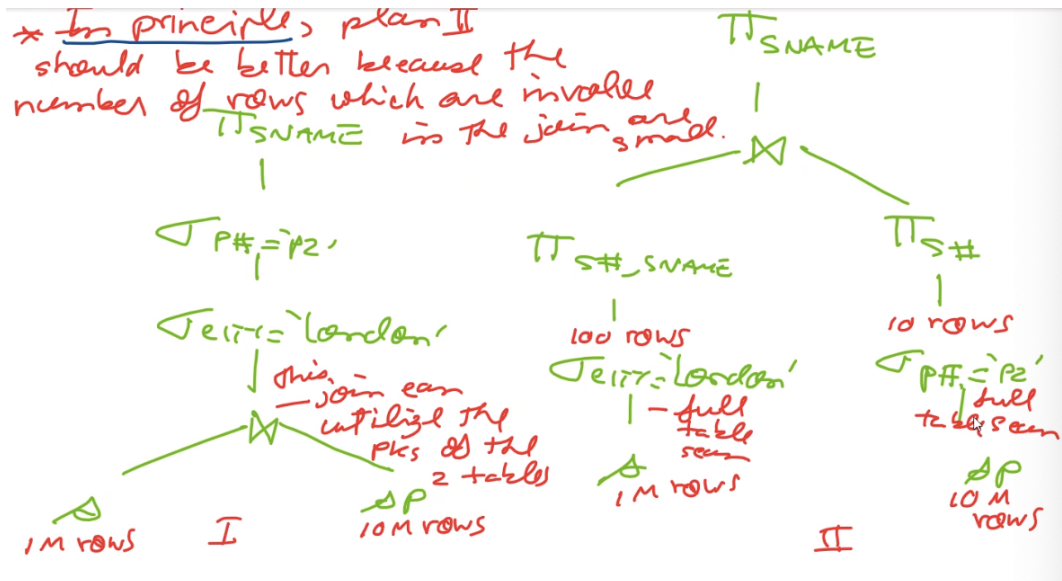
The dbms generate possible plan, and find the best one, it will try to reduce the number of row when using binary operation (join)

Example (2)



- primary key is logical, index is physical (not the same)
- to enforce, the uniqueness of primary key → the dbms usually create the unique index under primary key for uniqueness enforcement.
- we can said that for each primary key, there is an underlining unique index under the primary key) → first one will be join using index
- in the case, some don't have index???

- city and pnumber don't have indices (there combine index sp, you can utilize when you search both sp, or s but not p alone, because it from left to right)
- so no index → mean full table scan on city and p#.



index and full table scan thing

- full table scan will be slower → so plan number 1 will be used

It is possible that plan I is chosen to avoid 2 full table scans, so the choice by DBMS is now against the principle. (possible that join using index is slower than full table scan)

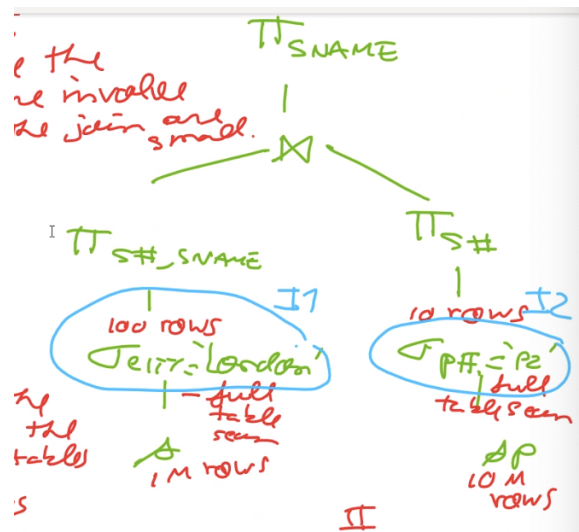
we expect the select first before the join, but because lack of indices

- we should trust dbms that they(optimizer) know best which way to go. → against principle, but because don't have index.

human programmer may help the optimizer, we could actually create the new index I1 on city column, create the new index i2 on p number.

- add this to statistics (after create might have to inform optimizer) (introduce to database statistic)

After having index, it's likely that the system will use plan number 2 and use i1 and i2



After the introduction of I1 index on city and I2 index on p number, the optimizer should prefer plan number 2 to plan number 1.

SQL statement

List SNAME of London suppliers who supply p2. (Please write at least 2 relational algebra statements)

- semi join type of question
- the output sname is from 1 table from s, but search condition is in another table.
- result from 1 table, search condition is on another table.
 - we can implement using join or subquery.

LIST SNAME of London suppliers who supply P2 (Join, subquery)

```

Select SNAME
From S natural join SP
where CITY = 'London' and P# = 'P2'

```

```

Select SNAME
From S
where CITY = 'London'
and S# in (Select S#
           From SP
           where P# = 'P2')

```

```

3) Select SNAME
    From S
    where CITY = 'London'
    and exists
      (Select *
       From SP
       where SP.S# = S.S#
       and P# = 'P2')

```

- natural join
- subquery
 - logically, we find all s number who supply p2.
- subquery with correlate??

Which one is faster?

- we don't know, it depend on relational algebra statment, not sql statement.
- somw dbms is so good, they can detect that this 3 statement is the same statement (give the same result → and therefore, share the same relational algebra statement)

Product by product.

Quick review

- Type of Query optimizer

1. rule-based optimizer

- this type of optimizer use fixed set of rule which normally depends on the way the input database statement is written.
 - except oracle 9.
- like the order of table in from clause, order of where condition
- this is actually the default optimizer that come with all dbms
 - easy to implement, but normally depends on the way the statement is written
 - not always depends on the way the statement is written
 - some year ago, the software tool read source code(in any language) as input, and provide optimize sql statement as output. The tool will scan your source code, and optimize your sql staement.
 - the tool assume that those source code input statement work on rule-based optimizer, because the internal cost-based optimizer will do the same thing → the produc tis kindof the external cost-based optimizer
 - tool → swap order of from, where, having cause, and may consider changing statement like subquery into where or join.
 - that tool know the order rule-based → and swap
 - the tool was useless for system that use cost-based optimizer.
 - the tool didn't work on oracle 9, when we use oracle 9, regardless of the way we uwrite sql statement → like rule-based optimizer in oracle 9 is dependant on the wy you write sql code.
 - so we change from oracle 9 to oracle 8, the rule-based optimizer depends on the way you wrote the statement.
 - oracle 9 choose a fixed plan.

2. cost-based optimizer

- chapter 15 consider cost based optimizer, not consider rule based at all
- it use database statistics which need to be pre collected to determine the best route. To obtains the result.
- in a year 2000, someone make review of many commercial dbms, and in that year all of them has cost-based optimizer and rule-based as a backup

3. semantic query optimizer

- uses integrity rules to help determine the best route. (Before using cost-based optimizer.)
- this is something on top of cost based optimizer
- like if you have a rule like the student can't borrow more than 5 books, rule encode in dbms.
- if the existing borrow record is not greater than 5, insert is allowed.
- in the case that user ask something like list student who borrow more than 10 books.
 - which semantic query optimzier tha query would search entire databse before returning not found.
- But if the system employ semantic query optimizer → the query will check against integrity rule → and it will know → not found will be answer straigh away.
- We should tell dbms which one we want to use.

The choice of query depends on the admin.

- typical command is set rbo on → rule-based optimizer (oracle)
 - when should we set rule-based optimizer to be on → when we don't have database statistic.
 - so ac hoc query (that you ask only once, and don't care much) → result may not be optimal, but the dbms don't have to think, it might be faster.
 - for some situation, maybe a better solution for ad hoc query.

- set cbo on → cost-based optimizer
 - database statistics must be available. because cost based use database statistics, even if you set cost-based optimizer on but you don't have database statistics, it gonna set back to rule based.
 - cost based optimizer give best solution, but take time to think. (analyze, plan)
- if oracle has semantic based?

This technology are available on a selected dbms. Not all have semantic optimizer.

Most have cost-based (you have to enable them)

- some times, you work on huge database, that you can't collect all database statistic for all table, you may choose to collect database statistics of some table only.
- in sql statement, you may have a query that refer to table with database statistics, and one without database statistics.

Select ...
 From T1, T2

— has DB statistics pre collected (under T1)
 — No DB statistics (under T2)

- the system will collect T2 database statistics on the fly.
- and it will take time
- in this case, its possible that using rule-based optimizer is a good choice.
- so you set rbo on → run the query.

So, at work, if 1 day you find out your query is maybe this case.

If T1, T2 both don't have db statistics and we set cost based

- depend on dbms
 - it may switch back to rule based straight away.
-